

```

64 var x = o.left;
65 var y = o.top;
66
67 var ax = settings.accX;
68 var ay = settings.accY;
69 var th = t.height();
70 var wh = w.height();
71 var tw = t.width();
72 var ww = w.width();
73
74 if (y + th + ay >= b &&
75     y <= b + wh + ay &&
76     x + tw + ax >= a &&
77     x <= a + ww + ax) {
78     //trigger the custom event
79     if (!t.appeared) t.trigger('appear', settings.data);
80
81     } else {
82     //it scrolled out of view
83     t.appeared = false;
84
85     }
86
87 };
88
89 //create a modified fn with some additional logic
90 var modifiedFn = function() {
91
92     //mark the element as visible
93     t.appeared = true;
94
95     //is this supposed to happen only once?
96     if (settings.one) {
97
98         //remove the check
99         w.unbind('scroll', check);
100         var i = $.inArray(check, $.fn.appear.checks);
101         if (i >= 0) $.fn.appear.checks.splice(i, 1);
102
103     }
104
105     //trigger the original fn
106     fn.apply(this, arguments);
107
108 };
109
110 //bind the modified fn to the element
111 $(settings.one) t.one('appear', settings.data, modifiedFn);
112 settings.data, modifiedFn);

```



MATERIA:

Programación Orientada a Objetos

Unidad 4 Persistencia, Archivos y Concurrencia Moderna

Docente:

Grupo:

Unidad #4 - Persistencia, Archivos y Concurrencia Moderna.

4.2.4. Lectura y escritura de objetos.

4.2.4.1. Serialización.

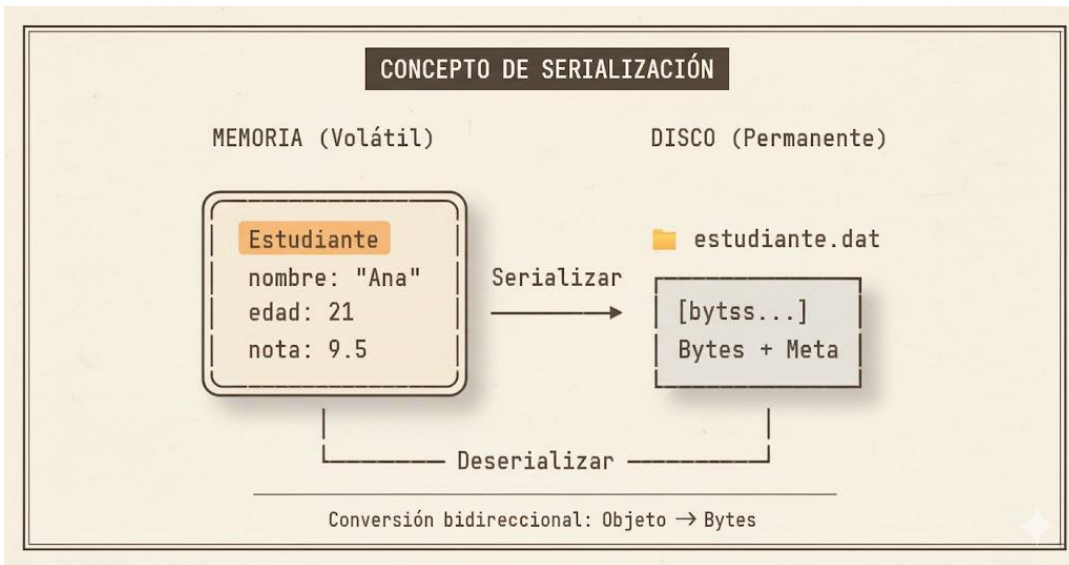
4.2.4.2. Creación de objetos ObjectOutputStream y ObjectInputStream.

4.2.4.3. Deserialización de objetos.

Lectura y escritura de objetos.

¿QUÉ ES LA SERIALIZACIÓN?

Proceso de convertir un objeto Java completo que existe en memoria RAM en una secuencia de bytes que puede ser almacenada o transmitida



Permite:

- ✓ Guardar objetos en archivos del disco duro
- ✓ Transmitir objetos a través de una red
- ✓ Almacenar objetos en bases de datos
- ✓ Crear copias exactas de objetos
- ✓ Implementar cachés persistentes

EL PROBLEMA DE LA VOLATILIDAD

¿Por qué necesitamos la serialización?

LA MEMORIA RAM ES VOLÁTIL:

Situación	¿Qué pasa con los objetos?	Resultado
● Cerrar el programa	Se borran	Pérdida total
● Apagar la computadora	Desaparecen	Pérdida total
● Error del sistema	Se pierden	Pérdida total
● Fallo eléctrico	Se destruyen	Pérdida total

LA SOLUCIÓN: SERIALIZACIÓN

Con Serialización	Beneficio
Datos en disco duro	Permanentes y seguros
Recuperación garantizada	Después de cerrar programa
Persistencia de estado	Objetos sobreviven apagado
Portabilidad	Transferir entre sistemas

APLICACIONES PRÁCTICAS

¿Dónde se usa en el mundo real?

VIDEOJUEGOS

Guardar progreso del jugador
Inventario de objetos
Puntuación y logros
Configuraciones personalizadas

APLICACIONES DE ESCRITORIO

- Preferencias del usuario
- Configuraciones de la aplicación
- Estado de ventanas y paneles
- Historial de acciones (deshacer/rehacer)

COMUNICACIÓN DISTRIBUIDA

- Enviar objetos entre cliente-servidor
- Comunicación entre aplicaciones Java (RMI)
- Mensajería entre sistemas
- Sincronización de datos

SISTEMAS DE CACHE

- Almacenar resultados de cálculos costosos
- Evitar reprocesamiento de datos
- Mejorar rendimiento de aplicaciones
- Reducir consultas a bases de datos

PERSISTENCIA DE SESIONES

- Mantener sesiones de usuario entre reinicios
- Guardar carritos de compra
- Datos temporales del usuario

INTERFAZ SERIALIZABLE

Requisito fundamental para serializar

```
import java.io.Serializable;

public class Estudiante implements Serializable {
    // Identificador de versión (MUY IMPORTANTE)
    private static final long serialVersionUID = 1L;

    // Atributos que SI se serializan
    private String nombre;
    private int edad;
    private double promedio;
    private String carrera;

    // Constructor
    public Estudiante(String nombre, int edad, double promedio) {
        this.nombre = nombre;
        this.edad = edad;
        this.promedio = promedio;
    }

    // Getters y setters...
}
```

Características de Serializable:

- Es una **interfaz marcadora** (marker interface)
- No tiene métodos que implementar
- Solo indica permiso para serialización
- Java la verifica en tiempo de ejecución
- Si falta, lanza **NotSerializableException**

¿Qué se serializa automáticamente?

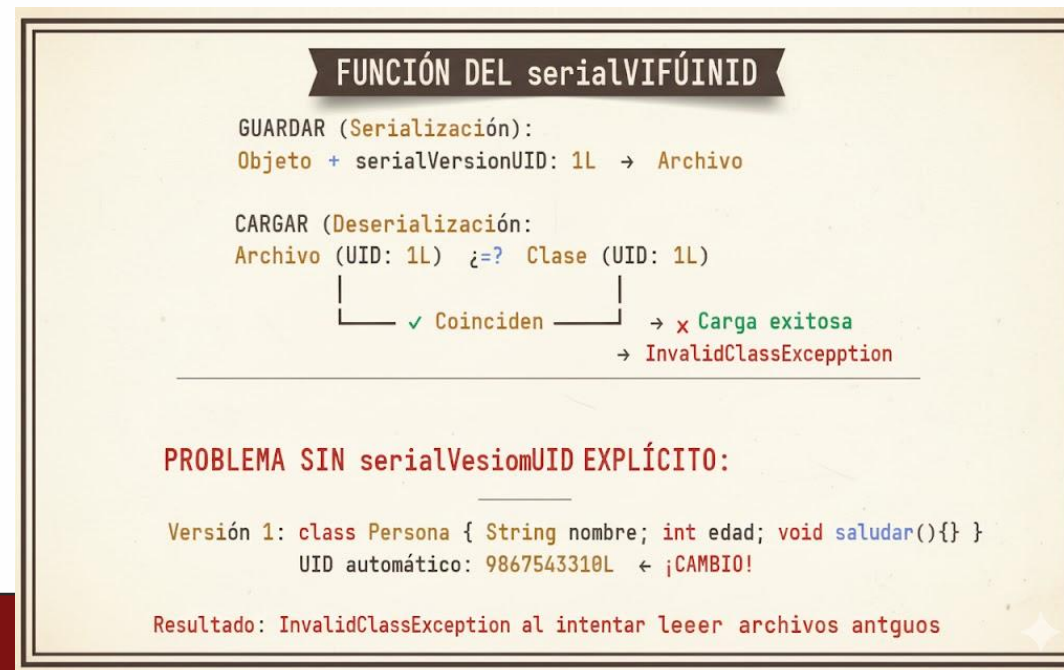
- ✓ Todos los atributos de instancia (no static)
- ✓ Objetos referenciados (si también son Serializable)
- ✓ Colecciones y arrays
- ✗ Métodos NO se serializan (solo datos)



```
private static final long serialVersionUID = 1L;
```

¿Qué es? Número de identificación único que actúa como "huella digital" de la clase

¿Cómo funciona?



- ✓ SIEMPRE declararlo explícitamente
- ✓ Usar 1L inicialmente
- ✓ Solo cambiar si quieres invalidar archivos viejos
- ✓ Incrementar versión cuando cambies estructura crítica

PALABRA CLAVE transient

Excluir campos de la serialización

```
java
import java.io.Serializable;

public class Usuario implements Serializable {
    private static final long serialVersionUID = 1L;

    // Campos que SÍ se serializan
    private String username;    // ✓ Se guarda
    private String email;      // ✓ Se guarda
    private int loginCount;     // ✓ Se guarda

    // Campos que NO se serializan (transient)
    private transient String password;    // ✗ NO se guarda
    private transient String sessionToken; // ✗ NO se guarda
    private transient long sessionId;     // ✗ NO se guarda
}
```

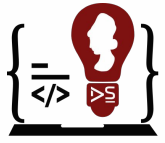
Elemento	Código/Palabra Clave	Función
Permiso	<code>implements Serializable</code>	Otorga permiso a la JVM para guardar el objeto.
Compatibilidad	<code>serialVersionUID = 1L</code>	Control de Versiones. Asegura que los archivos viejos (<code>1L</code>) solo puedan ser leídos por la clase que tiene el mismo ID, previniendo la excepción <code>InvalidClassException</code> .
Exclusión	<code>transient</code>	NO guarda el campo en el archivo. Se usa para datos sensibles (<code>password</code>) o temporales. Al cargar el objeto, el campo queda con valor por defecto (<code>null</code>).
Inclusión	Campos normales	Por defecto, todos los demás campos de instancia (<code>username</code> , <code>email</code> , etc.) SÍ se guardan y se restauran.

¿Cuándo usar transient?

Categoría	Ejemplos	Razón
 Seguridad	Contraseñas, tokens, claves API	Datos sensibles no deben guardarse
 Temporal	IDs de sesión, timestamps actuales	Información válida solo en ejecución
 Recursos	Conexiones DB, archivos abiertos	No pueden serializarse
 Calculable	Totales, índices derivados	Se pueden recalcular después

Ejemplo práctico:

```
class CuentaBancaria implements Serializable {  
    private double saldo;           // ✓ Se guarda  
    private transient Connection dbConnection; // × Recurso no serializable  
    private transient double interesAcumulado; // × Se recalcula al cargar  
}
```



Ingeniería en
**DESARROLLO
DE SOFTWARE**

CAMPOS static - NUNCA SE SERIALIZAN

Los campos static pertenecen a la CLASE, no a instancias

```
public class Contador implements Serializable {  
    private static final long serialVersionUID = 1L;  
  
    // Campo STATIC - pertenece a la clase  
    private static int contadorGlobal = 0; // ✗ NO se serializa  
  
    // Campo de INSTANCIA - pertenece al objeto  
    private int contadorInstancia; // ✓ SÍ se serializa  
  
    public Contador() {  
        contadorGlobal++;  
        contadorInstancia = contadorGlobal;  
    }  
}
```

Ejemplo en acción

CREAR 3 OBJETOS:

c1, c2, c3

contadorGlobal = 3 (compartido por todos)

GUARDAR c2:

Solo se guarda: contadorInstancia = 2

NO se guarda: contadorGlobal

REINICIAR PROGRAMA Y CARGAR:

contadorGlobal = 0

+ Se reinició (valor por defecto)

contadorInstancia = 2

+ Se restauró del archivo

ObjectOutputStream

ESCRIBIR OBJETOS

Clase especializada para serialización

```
// Crear el stream de salida
ObjectOutputStream oos = new ObjectOutputStream(
    new FileOutputStream("archivo.dat")
);
```

Este fragmento sigue el **Patrón Decorator** :

- **FileOutputStream("archivo.dat")** es el stream base (el *decorado*). Define el **destino** físico (el archivo en disco) donde se escribirán los bytes.
- **ObjectOutputStream(fos)** es el stream *decorador*. **Añade la funcionalidad** clave: la capacidad de tomar objetos Java y convertirlos en bytes serializados antes de entregarlos al **FileOutputStream** para que este los escriba al disco.

ObjectOutputStream

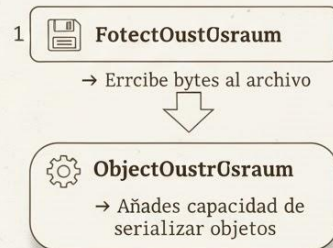
MÉTODOS PRINCIPALES Y PATRÓN DECORATOR

Métodos principales de "ObjectOustrOsraum":

Método	Descripción	Ejemplo
writeObject(Object obj)	Serializa objeto completo	oss.writeObject(oss.estudiante);
writeInt(int v)	Escribe un entero	
writeUnit(entero)	EscribeDouble(dauble v)	oss.writeInt(42);
writeDouble(dauble v)	Escribe String en UTF-8	writeUTF("Hola");
Escribe String en UTF-8	writeUTF(String true);	
writeBoolean	oss.flush()	oss.flush();
flush()	Forza salida inmediata	
close()	Cierra y libera recursos	oss.close();

Patrón Decorator en Streams:

↓ envuelto por



Ventaja: Cada capa añade funcionalidad sin alterar la original.

MUY IMPORTANTE:

- Siempre cerrar el stream con `close()`
- Usar try-with-resources para cierre automático
- Los datos pueden quedar en buffer sin `close()`

EJEMPLO ESCRITURA COMPLETO

Guardar un objeto en archivo

```
import java.io.*;

public class EjemploEscritura {
    public static void guardarEstudiante() {
        // try-with-resources: cierra automáticamente
        try (ObjectOutputStream oos = new ObjectOutputStream(
            new FileOutputStream("estudiante.dat"))) {

            // Crear el objeto
            Estudiante est = new Estudiante("Ana García", 21, 9.5);

            // Serializar y guardar
            oos.writeObject(est);

            System.out.println("✓ Estudiante guardado exitosamente");

        } catch (IOException e) {
            System.err.println("Error al guardar: " + e.getMessage());
            e.printStackTrace();
        }
    }
}
```

Qué pasa internamente?

1. Se crea el archivo "estudiante.dat"
2. ObjectOutputStream analiza el objeto
3. Convierte atributos a bytes
4. Escribe metadatos (clase, serialVersionUID)
5. Escribe los datos serializados
6. try-with-resources cierra automáticamente

Archivo generado: Binario, no legible con editor de texto

ObjectInputStream - LEER OBJETOS

Clase especializada para deserialización

```
java

// Importaciones necesarias
import java.io.*;

// Crear el stream de entrada
ObjectInputStream ois = new ObjectInputStream(
    new FileInputStream("archivo.dat")
);
```

Stream Base `new FileInputStream("archivo.dat")` Define el **origen** de los bytes: el archivo (`archivo.dat`) que contiene los datos serializados.

Stream Decorador	<code>ObjectInputStream(fis)</code>	Añade la capacidad de Deserialización. Toma los bytes leídos por <code>FileInputStream</code> y los transforma de nuevo en el Objeto Java original en memoria.
-------------------------	-------------------------------------	--

`ObjectInputStream` lee del archivo y usa su método clave, `readObject()`, para reconstruir el objeto. Este proceso debe ejecutarse en el **mismo orden** en que fueron escritos los datos.

ObjectInputStream

Métodos principales:

Método	Descripción	Retorno	Ejemplo
<code>readObject()</code>	Lee y deserializa objeto	Object (requiere cast)	<code>(Estudiante) ois.readObject()</code>
<code>readInt()</code>	Lee un entero	int	<code>int edad = ois.readInt()</code>
<code>readDouble()</code>	Lee un double	double	<code>double nota = ois.readDouble()</code>
<code>readUTF()</code>	Lee String en UTF-8	String	<code>String nombre = ois.readUTF()</code>
<code>readBoolean()</code>	Lee un boolean	boolean	<code>boolean activo = ois.readBoolean()</code>
<code>close()</code>	Cierra y libera recursos	void	<code>ois.close()</code>

REGLA CRÍTICA: Los datos deben leerse en el **MISMO ORDEN** en que se escribieron

```
java
// ESCRITURA:
ois.writeInt(42);
ois.writeUTF("Ana");

// LECTURA (mismo orden):
int numero = ois.readInt();    // ✓ Correcto
String nombre = ois.readUTF(); // ✓ Correcto

// ✗ ERROR:
String nombre = ois.readUTF(); // ✗ Lee bytes de int como String
int numero = ois.readInt();    // ✗ Desincronizado
```

	Escritura (Serialización)	Lectura (Deserialización)
Orden	<code>writeInt()</code> luego <code>writeUTF()</code>	Debe ser <code>readInt()</code> luego <code>readUTF()</code>
Principio	Lo que se escribe primero...	...se lee primero
Resultado	La lectura debe coincidir el Orden y el Tipo de Dato escrito.	
Fallo	Intentar leer un <code>String</code> cuando se escribió un <code>int</code> .	Desincronización → Lanza <code>IOException</code> .

EJEMPLO LECTURA COMPLETO

```
import java.io.*;

public class EjemploLectura {
    public static void cargarEstudiante() {
        try (ObjectInputStream ois = new ObjectInputStream(
            new FileInputStream("estudiante.dat"))) {

            // Deserializar: leer y convertir bytes a objeto
            // Requiere CAST porque readObject() retorna Object
            Estudiante est = (Estudiante) ois.readObject();

            // Usar el objeto restaurado
            System.out.println("Nombre: " + est.getNombre());
            System.out.println("Edad: " + est.getEdad());
            System.out.println("Promedio: " + est.getPromedio());

        } catch (FileNotFoundException e) {
            System.err.println("Archivo no encontrado");
        } catch (IOException e) {
            System.err.println("Error al leer: " + e.getMessage());
        } catch (ClassNotFoundException e) {
            System.err.println("Clase Estudiante no encontrada");
        }
    }
}
```

Este código en Java **lee (deserializa)** un objeto guardado previamente en un archivo llamado **"estudiante.dat"** y muestra su información en consola.

♦ Qué hace paso a paso:

1. Usa **ObjectInputStream** para abrir y leer el archivo binario.
2. Convierte (CAST) los datos leídos a un objeto de tipo **Estudiante**.
3. Imprime el **nombre, edad y promedio** del estudiante.
4. Controla errores comunes: archivo no encontrado, error de lectura o clase no disponible.

En resumen: **recupera un objeto Estudiante desde un archivo y muestra sus datos.**

Excepciones a manejar

`**FileNotFoundException**` → El archivo no existe

`**IOException**` → Error de lectura del archivo

`**ClassNotFoundException**` → La clase no está en el classpath

`**InvalidClassException**` → serialVersionUID no coincide

PROCESO DE DESERIALIZACIÓN

De bytes a objeto: pasos internos



SERIALIZACIÓN: Objeto → Bytes
Bytes → Objeto DESERIALIZACIÓN:

CONSTRUCTOR NO SE EJECUTA

DIFERENCIA CRÍTICA CON CREACIÓN NORMAL

Aspecto	CREACIÓN NORMAL <code>new</code>	DESERIALIZACIÓN <code>readObject()</code>
Llamada a constructor	✓ Sí se ejecuta	✗ NO se ejecuta
Validación de parámetros	✓ Se validan	✗ NO se validan
Inicialización de campos	✓ Por constructor	• Directamente desde bytes
Lógica de negocio	✓ Se ejecuta	✗ NO se ejecuta
Efectos secundarios	✓ Se producen	✗ NO ocurren

Ejemplo del problema:

```
public class CuentaBancaria implements Serializable {
    private double saldo;

    public CuentaBancaria(double saldoInicial) {
        // ✓ VALIDACIÓN en constructor
        if (saldoInicial < 0) {
            throw new IllegalArgumentException("Saldo no puede ser negativo");
        }
        this.saldo = saldoInicial;

        // ✓ Lógica de negocio
        registrarEnSistema();
        enviarNotificacion();
    }

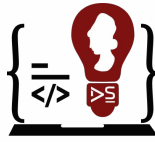
    // CREACIÓN NORMAL:
    new CuentaBancaria(-100); // ✓ Lanza excepción, válida

    // DESERIALIZACIÓN:
    ois.readObject(); // ✗ NO válida, saldo puede ser negativo
                    // ✗ NO llama registrarEnSistema()
                    // ✗ NO envía notificación
}
```

Este código muestra que al **deserializar un objeto (`readObject()`)** de la clase `CuentaBancaria`, **no se ejecuta el constructor**, por lo que:

- **No se valida** si el saldo es negativo.
- **No se ejecuta** la lógica de negocio (`registrarEnSistema()` ni `enviarNotificacion()`).

👉 En resumen: la **deserialización restaura los datos directamente** sin pasar por las validaciones ni acciones del constructor.



Ingeniería en DESARROLLO DE SOFTWARE

COMPORTAMIENTO DE transient

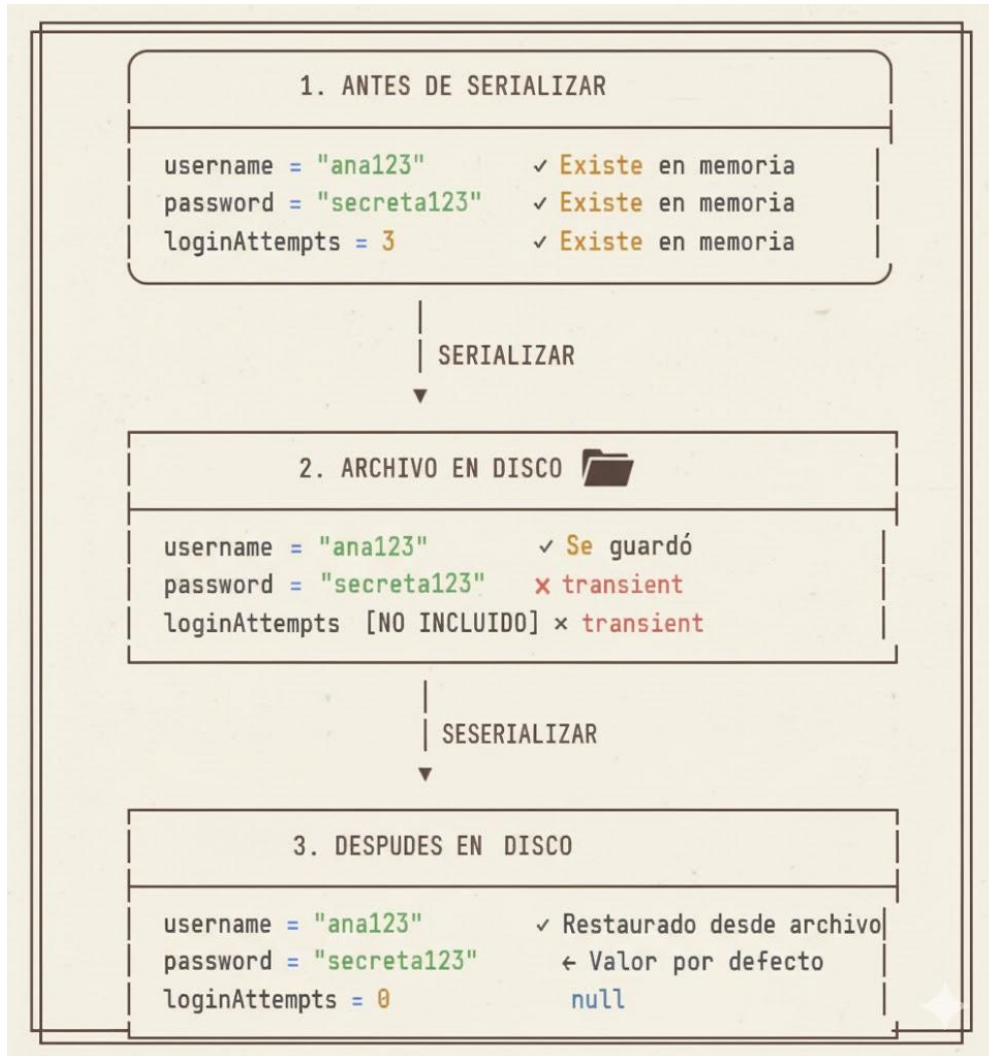
¿Qué pasa con campos transient al deserializar?

```
class Usuario implements Serializable {  
    private String username;           // Campo normal  
    private transient String password; // Campo transient  
    private transient int loginAttempts; // Campo transient  
}
```

Explicación corta:

- La palabra clave **transient** indica que esos atributos **no se incluyen en la serialización**.
- Así, al deserializar, **password** y **loginAttempts** quedarán en **valores por defecto** (**null** y **0**).
- Se usa para **proteger datos sensibles** o **temporales** que no deben almacenarse en disco.
Serializable permite guardar objetos,
transient excluye ciertos campos de ese proceso.

Ciclo completo



VALORES POR DEFECTO EN DESERILIZACIÓN

Valores por defecto según tipo:

TIPO DE DATO	VALOR POR DEFECTO
byte, short, int, long	0
float, double	0.0
boolean	false
boolean	"\u0000" (Nulo)
Object (referencias)	null

Solución si necesitas el valor:
Recalcular o reinicializar en un
método post-deserilización 

IMPORTANCIA DE CERRAR STREAMS

Qué pasa si NO cierras el stream?

PROBLEMA 1: DATOS INCOMPLETOS

- Los últimos datos quedan en memoria (buffer)
- NUNCA se escriben al disco
- Archivo queda corrupto o incompleto
- Al intentar leer: `IOException`

PROBLEMA 2: FUGA DE RECURSOS

- El sistema operativo mantiene el archivo abierto
- Límite de archivos abiertos se agota
- Eventualmente: "Too many open files"
- La aplicación puede crashear

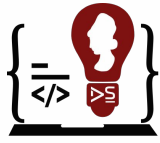
PROBLEMA 3: ARCHIVO BLOQUEADO

- El archivo permanece bloqueado
- Otras aplicaciones no pueden acceder
- No se puede borrar o renombrar
- Problemas de concurrencia

✓ SOLUCIÓN: try-with-resources

```
java
// ✗ MAL - Puede no cerrar si hay excepción:
ObjectOutputStream oos = new ObjectOutputStream(...);
oos.writeObject(objeto);
oos.close(); // Si hay error antes, nunca se ejecuta

// ✓ BIEN - Cierre automático garantizado:
try (ObjectOutputStream oos = new ObjectOutputStream(...)) {
    oos.writeObject(objeto);
} // close() se llama automáticamente, incluso con excepciones
```

Ingeniería en
**DESARROLLO
DE SOFTWARE**

VALIDACIÓN POST-DESERIALIZACIÓN

SIEMPRE validar datos después de cargar

```
public class ValidacionSegura {

    public static Estudiante cargarYValidar(String archivo)
        throws IOException, ClassNotFoundException {

        // Cargar objeto
        Estudiante est;
        try (ObjectInputStream ois = new ObjectInputStream(
            new FileInputStream(archivo))) {
            est = (Estudiante) ois.readObject();
        }

        // ✅ VALIDAR CAMPOS OBLIGATORIOS
        if (est.getNombre() == null || est.getNombre().isEmpty()) {
            throw new IllegalStateException(
                "Nombre inválido o vacío");
        }

        // ✅ VALIDAR RANGOS NUMÉRICOS
        if (est.getEdad() < 0 || est.getEdad() > 150) {
            throw new IllegalStateException(
                "Edad fuera de rango válido: " + est.getEdad());
        }

        // ✅ VALIDAR VALORES LÓGICOS
        if (est.getPromedio() < 0.0 || est.getPromedio() > 10.0) {
            throw new IllegalStateException(
                "Promedio inválido: " + est.getPromedio());
        }

        // ✅ VALIDAR CONSISTENCIA
        if (est.getCarrera() == null) {
            est.setCarrera("No especificada");
        }

        return est;
    }
}
```

QUÉ VALIDAR EN LA DESERIALIZACIÓN

Categorías de Validación

Categoría	Ejemplos	Razon
⊗ Nulls	Campos obligatorios no null	Prevenir NullPointerException
📏 Rangos	Edad 0-150, notas 0-10	Datos lógicamente válidos
@ Formatos	Emails, URLs, fechas	Integridad de datos
🔗 Consistencia	Relaciones entre campos	Coherencia lógica
↔️ Tamazos	Strings no vacíos, listas	Prevenir estados inválidos

! ¡ESPECIALMENTE CRÍTICO: Archivos de fuentes externas o no confiables

SEGURIDAD EN DESERIALIZACIÓN

VULNERABILIDADES DE SEGURIDAD

RIESOS DE SEGURIDAD EN LA DESERIALIZACIÓN

EJECUCIÓN DE CÓDIGO MALICIOSO

- Objetos con código malicioso en métodos
- Ataques de "gadget chains"
- Puede ejecutar comandos del sistema

CONSUMO EXCESIVO DE RECURSOS

- Objetos gigantes (DOS - Denial de Service)
- Estructuras circulares infinitas
- Agotar memoria RAM

INYECCIÓN DE OBJETOS PELIGROSOS

- Objetos con privilegios elevados
- Bypass de validaciones de seguridad
- Escalada de privilegios

MEDIDAS DE PREVENCIÓN

Medida	Implementación	Efectividad
Nunca deserializar fuentes no confiables	Rechazar archivos externos	★★★★★
Validar TODO después de leer	Método validar() obligatorio	★★★★
Usar JSON para datos externos	Jackson, Gson en lugar de serialización	★★★★★
Limitar tamaño de archivos	Máximo 1MB por archivo	★★★
Lista blanca de clases	Solo deserializar clases conocidas	★★★★
Sandboxing	Ejecutar en entorno aislado	★★★★★

Ejemplo de filtro de clases:

```
java
// Implementar ObjectInputFilter (Java 9+)
ObjectInputFilter filter = info -> {
    if (info.serialClass() == Estudiante.class) {
        return Status.ALLOWED;
    }
    return Status.REJECTED;
};
ois.setObjectInputFilter(filter);
```

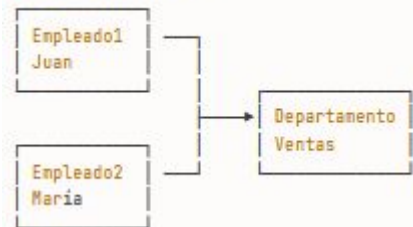
REGLA DE ORO: La serialización Java es para datos INTERNOS de confianza.
Para datos EXTERNOS, usar JSON o XML.

Java mantiene relaciones entre objetos correctamente

```
class Departamento implements Serializable {  
    private String nombre;  
}  
  
class Empleado implements Serializable {  
    private String nombre;  
    private Departamento depto; // Referencia  
}
```

Escenario

ANTES DE SERIALIZAR:



Empleado1 y Empleado2 apuntan al MISMO objeto
Departamento

DESPUÉS DE DESERIALIZAR

- ✓ Sigue siendo UN SOLO objeto Departamento
- ✓ Empleado1 y Empleado2 apuntan al mismo
- ✓ Referencias se preservan correctamente
- ✓ Sin duplicación de objetos

Mecanismo interno: Java asigna IDs únicos a cada objeto durante serialización:

- Departamento: ID #1
- Empleado1: ID #2 (referencia a #1)
- Empleado2: ID #3 (referencia a #1)

Al deserializar, reconstruye la estructura completa con las mismas referencias

Ventaja: Grafos complejos de objetos se mantienen intactos

EJEMPLO COMPLETO - LISTA

Serializar y deserializar colecciones

```
import java.io.*;
import java.util.*;

public class EjemploCompleto {

    // GUARDAR lista de estudiantes
    public static void guardar(List<Estudiante> lista, String archivo) {
        try (ObjectOutputStream oos = new ObjectOutputStream(
            new FileOutputStream(archivo))) {

            oos.writeObject(lista);
            System.out.println("✓ " + lista.size() + " estudiantes guardados");

        } catch (IOException e) {
            System.err.println("Error al guardar: " + e.getMessage());
        }
    }

    // CARGAR lista de estudiantes
    public static List<Estudiante> cargar(String archivo) {
        try (ObjectInputStream ois = new ObjectInputStream(
            new FileInputStream(archivo))) {

            @SuppressWarnings("unchecked")
            List<Estudiante> lista = (List<Estudiante>) ois.readObject();

            // VALIDAR cada estudiante
            for (Estudiante est : lista) {
                if (est == null || est.getNombre() == null) {
                    throw new IllegalStateException("Datos corruptos");
                }
            }

            System.out.println("✓ " + lista.size() + " estudiantes cargados");
            return lista;

        } catch (FileNotFoundException e) {
            System.out.println("Archivo no existe, lista vacía");
            return new ArrayList<>();
        } catch (IOException | ClassNotFoundException e) {
            System.err.println("Error: " + e.getMessage());
            return new ArrayList<>();
        }
    }
}
```

```
// MAIN de prueba
public static void main(String[] args) {
    String archivo = "estudiantes.dat";

    // Crear y guardar
    List<Estudiante> lista = new ArrayList<>();
    lista.add(new Estudiante("Ana", 21, 9.5));
    lista.add(new Estudiante("Luis", 22, 8.7));
    lista.add(new Estudiante("Maria", 20, 9.8));
    guardar(lista, archivo);

    // Cargar
    List<Estudiante> cargados = cargar(archivo);
    cargados.forEach(System.out::println);
}
```

- **Guardar (guardar):** Utiliza `ObjectOutputStream` para tomar la lista de objetos en memoria y escribirla como bytes en un archivo (`oos.writeObject(lista)`).
- **Cargar (cargar):** Utiliza `ObjectInputStream` para leer los bytes del archivo y reconstruir la lista de objetos (`ois.readObject()`).
- **Validación (Crítica):** El método `cargar` incluye un bucle para **validar cada objeto** cargado. Esto es vital porque la deserialización no ejecuta constructores, permitiendo que datos corruptos (como un nombre nulo) se cuelen, lo cual podría causar fallos (`IllegalStateException`).
- **Manejo de Errores:** Captura excepciones comunes de I/O (`IOException`) y de compatibilidad de clases (`ClassNotFoundException`) para manejar problemas de archivos.